



Running Commands The Pipeline Providers

Running Commands

Good ol' Command Line

- Just because we can use PS to Script doesn't mean we must
- We can make scripts from commands, but Powershell is both simple enough to a functional Command Line and intricate enough to be a scripting language.

Elements of a Command

- Full-Form Syntax: a command that has not be abbreviated in any way.

```
New-Item -Type File -Path C:\Scripts\file1.txt,C:\Scripts\file2.txt -whatif
```

↑
cmdlet

↑
Param 1

↑
Param 1
(value)

↑
Param 2

↑
Param 2
(multi-value)

↑
Switch
(Param 3
no value)

Terminology (Naming Conventions)

- CMDLET (*command-let*)
 - A “command” in PS
 - Usually written in .net or C#
- Function
 - Like cmdlet, but written in PS Script
- Workflow
 - Running sequences of long-running tasks
 - Affect multiple managed computers at the same time
 - Special function; uses PS Workflow execution
 - DevOps!

Terminology (Naming Conventions)

- Application
 - An external executable
- Command
 - Generic term for all of the above.

The Rules

- Cmdlets
 - Start with a verb (get, set, add)
 - End with a noun (Process, EventLog, Item)
 - Format: **Verb-Noun**
 - **Get-Verb** to find available verbs
- Why all the rules?
 - Consistent naming conventions allow you to *intuit* what command names will be (using **help** or **Get-Command**)

Aliases (nicknames)

- `Get-Alias -Definition <cmdlet>`
 - Gets the Alias for a cmdlet
- `Get-Alias -Name <alias>`
 - Gets the cmdlet for the Alias
- Many Alias mirror other command lines.

`DIR, LS` → `Get-ChildItem`

`Del, Erase` → `Remove-Item`

`Cat` → `Get-Content`

`CD, Chdir` → `Set-Location`

Creating Aliases

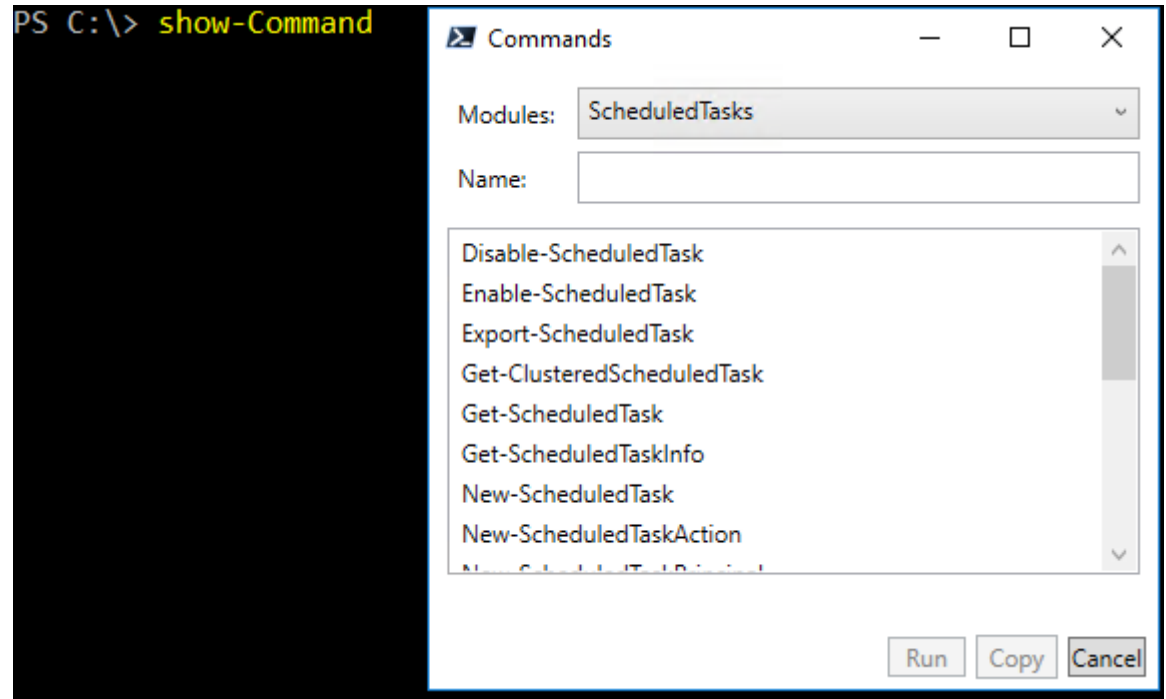
- When creating aliases
 - You can only create alias for cmdlets (not cmdlets + parameters, that would be a *function*)
 - Custom aliases only persist in the current PS session.
- `New-Alias -name <alias name> -value <cmdlet>`
- Example:
- `New-Alias -name go -value Set-Location`
- `Get-Alias -name go | Format-List`

Shortcuts (abbreviation)

- Truncating Parameters
 - You only need to type enough of the parameter name so that it is *unambiguous*.
 - Some Parameters have their own aliases.
Use `get-help -full` to find them.
- How would we shorten?
- `New-Alias -name go -value Set-Location`
- `Get-Alias -name go | Format-List`

Show-Command

- Allows you to use the Command window from ISE in a regular Powershell session.
- Extremely helpful when figuring out syntax for the first time.
- Separates Parameters Sets into tabs (so you can't mix them up).
- Once you populated the parameters, click run, or copy
- You can only work with one command at a time.



External Commands

- Most (if not all) commands, from other resident CLIs will work in Powershell (CMD for example).
- PING, IPCONFIG, etc all *still work*. These aren't aliases, they are CMD commands running in PS (so an *application*).
- These can fail if you use too many parameters.

Errors

- Error messages in PS include the Line number and Character at which a command has failed.
- Ensure you READ them, and investigate what they are telling you
- If you can decipher the message, use the commands associated help file to figure it out.
- Still can't figure it out? Google it.

The pipeline

What is the pipeline?

- Powershell connects commands to each other using the pipeline
- We use `|` to *pass* (pipe) the output from one command to another command to be processed.
- The implementation of `|` in Powershell is *much* more featureful than in other CLIs. A few examples....
 - Exporting output: `Get-Process | Export-CSV gci.csv`
 - Redirecting output: `Get-ChildItem | Out-File gci.txt`
 - Formatting output: `Get-ChildItem | Format-List`
 - Sorting output: `Get-ChildItem | Sort-Object -property length -descending`
 - Filtering output: `Get-ChildItem | where-Object {$_.Length -gt 1000}`

What is a Comma Separate Value (CSV) File?

- You may have noticed that there are a lot of tables in Powershell
- CSV are a way to denote a table (spreadsheet) as a text file that has the .CSV extension.
- Formatting:
 - All Items Separated by a Comma
 - No Spaces (Except in Cell Value, surrounded by quotes).
 - The First line of the CSV will denote the Table Headers (follows the same format as above)

.CSV Example

Name,Species,Diet
"Sir Dogingston",Dog,Omnivore
Meowzer,Cat,Carnivore

Name	Species	Diet
Sir Dogingston	Dog	Omnivore
Meowzer	Cat	Carnivore

Exporting to CSV/Importing CSVs

(all the tables)

- Most Get commands will successfully export its output to a CSV.
 - `Get-Process | Export-CSV -path gci.csv`
- Opening the .csv file with a text editor is not very helpful. CSVs are meant to be imported into a spreadsheet application (excel).
- Notice that, when output is redirected to CSV, *all* information for the current command is sent to the file (more than what would appear on the screen).
 - On screen output is formatted for the screen and will likely omit less relevant data. Piping to a CSV wont.
(more on formatting output on the screen later)
- That CSV file can then be imported and interpreted by another Powershell console by using `Import-CSV`.

Exporting to XML

- Like CSVs, XMLs are meant to contain data in a common exchange format.
(CSV and XML are referred to as a **Common Data Exchange Format**)
- XML files can be opened with an XML viewer or a web browser.
- Many applications use them as Import Data.

Why Export data to a Command Exchange Format?

- Outputting data to a static source is good for taking data comparisons. Start with *baseline*, make some changes, then compare the results.
“what the data looks like now, and what the data looks like after”
- When data is in the same format, we can compare it for differences/changes/deltas.
- **Compare-Object** (**diff**) can be used to examine two files and compare them.

Compare-Object

- `-referenceObject`
 - The input to be referenced
- `-differenceObject`
 - Check here to see what's different from the referenceObject.
- `-property`
 - Don't show us everything, parse out only the property I want.
(if you're not comparing a text file, line by line, you will want to use this command)

Changing your Output

- Powershell can output to more than just the console
- Out-Default (Out-Host): what PS does by default.
- Out-File:
 - Sends the output of a command to a file
 - Special: Can change the encoding of the file (its better than >)
- Out-Null: sends the output nowhere.
- OutGridView (if ISE and ,net3.5 are installed):
 - Sends the output to a WPF Grid view.
- Out-Printer
 - Sends the output to a PrintDevice

ConvertTo-HTML

- Different that the **Export** Commands
- Do not create a file, the **ConvertTo-*** commands convert the output to the correct format and display it to the Screen.
- Used with **Out-File**, we can create files based on the output.

Confused?

- Use **Get-Content** for text files.
- Use the appropriate **Import-*** command for that data exchange format.
- Compare-Object is not great at comparing text files with more than one line that's different.

Managing Services and Process

(a practical application of piping)

- Commands that share a noun can often pipe their output each other (you can pipe commands that *do not* share a noun to each other, but more on this later).

```
Get-Process -name Notepad.exe | Stop-Process
```

- The results of these commands can have system-wide impact (like managing Services and Processes)
- The `$ConfirmPreference` environment variable protects your environment from running 'high impact' commands.
 - Set to HIGH by default
 - If the commands 'impact level' is higher than the `$ConfirmPreference` level, you will be prompted for confirmation.
 - `-Confirm` and `-Whatif` are relevant here.

Providers

What *is* a Provider?

- A provider is a “data store” that “looks like” a disk drive.

```
PS C:\Users\Administrator> Get-PSPProvider
```

Name	Capabilities	Drives
----	-----	-----
Registry	ShouldProcess	{HKLM, HKCU}
Alias	ShouldProcess	{Alias}
Environment	ShouldProcess	{Env}
FileSystem	Filter, ShouldProcess, Credentials	{C, Temp, D, E}
Function	ShouldProcess	{Function}
Variable	ShouldProcess	{Variable}

- Providers can be added to the Shell VIA external modules or Snap-ins.

What *is* a Provider?

- Providers have “capabilities”, which define the way we interface with the Provider.
 - **ShouldProcess** – cmdlet supports the use of `-whatif` and `-confirm` `parameters` (to test a command prior to executing it)
 - **Filter** – cmdlet supports the use of the `-filter` parameter
 - **Credentials** – allows credentials (authentication) to be used when executing the cmdlet
 - **Transactions** – cmdlet supports transactions (make several changes, then commit or roll them back).
- PSProvider will “adapt” the datastore so that it can work with it.
- `Get-PSDrive` will list the available datastores on the local machine.
(`*-PSDrive` is what makes the Providers available for use)

What *is* a Provider?

- Most cmdlets for manipulating a provider include the noun **Item**.

```
PS C:\> Get-Command -noun *Item* | where {$_.CommandType -eq 'cmdlet'}
```

CommandType	Name	Version	Source
-----	----	-----	-----
Cmdlet	Clear-Item	7.0.0.0	Microsoft.PowerShell.Management
Cmdlet	Clear-ItemProperty	7.0.0.0	Microsoft.PowerShell.Management
Cmdlet	Copy-Item	7.0.0.0	Microsoft.PowerShell.Management
Cmdlet	Copy-ItemProperty	7.0.0.0	Microsoft.PowerShell.Management
Cmdlet	Get-ChildItem	7.0.0.0	Microsoft.PowerShell.Management
Cmdlet	Get-Item	7.0.0.0	Microsoft.PowerShell.Management
Cmdlet	Get-ItemProperty	7.0.0.0	Microsoft.PowerShell.Management
Cmdlet	Get-ItemPropertyValue	7.0.0.0	Microsoft.PowerShell.Management
Cmdlet	Invoke-Item	7.0.0.0	Microsoft.PowerShell.Management
Cmdlet	Move-Item	7.0.0.0	Microsoft.PowerShell.Management
Cmdlet	Move-ItemProperty	7.0.0.0	Microsoft.PowerShell.Management
Cmdlet	New-Item	7.0.0.0	Microsoft.PowerShell.Management
Cmdlet	New-ItemProperty	7.0.0.0	Microsoft.PowerShell.Management
Cmdlet	Remove-Item	7.0.0.0	Microsoft.PowerShell.Management
Cmdlet	Remove-ItemProperty	7.0.0.0	Microsoft.PowerShell.Management
Cmdlet	Rename-Item	7.0.0.0	Microsoft.PowerShell.Management
Cmdlet	Rename-ItemProperty	7.0.0.0	Microsoft.PowerShell.Management
Cmdlet	Set-Item	7.0.0.0	Microsoft.PowerShell.Management
Cmdlet	Set-ItemProperty	7.0.0.0	Microsoft.PowerShell.Management

Clarification on terminology

- Providers use different names to represent hierarchies in Windows.
- Cmdlets used with providers are *generifed* because they can be used with different providers (File system/Registry/etc.): Everything is an “ITEM”
- Items can have **properties** (some don't).
 - Read-Only
 - Length
 - LastWriteTime

File System	Registry	Providers
Drive	Hive (rootkey)	Drives
Folder	Keys	Items
File	Values	ChildItem (can be a leaf object)

Filesystem Navigation

- **Set-Location**

- Aliases are **CD**, **CHDIR**, **SL**
- Changes your PWD in the Filesystem
- Example:
**Set-Location -path ** (this changes to root)
Set-Location -path HKLM: (changes the current drive to HKCU and uses the registry provider.

- **New-Item**

- Aliases are **ni**, **mkdir**, **md**
- Creates a new item in the specified *drive*.
- Mandatory: **-type** (specifies the type of item to be created)
 - File, Directory, SymbolicLink, etc ← These types will be relative to the Provided being used.

-Path vs -LiteralPath

- Most commands that work with PSDrives support the `-path` parameter
- Wildcards can be used in the path parameter to search for specific items (exactly like DIR).
 - * = any character, any number of characters
 - ? = any SINGLE character.
- `Get-ChildItem -path *.log` (finds log files in the PWD).
- `Get-ChildItem -path ????.log` (finds any .log file with a 4-character name)
- Use `-path` when Wildcards are NOT allowed in file names
- Use `-LiteralPath` when wildcards are legal characters in a filename

Working with Registry (changing a value)

1. Change to the Registry (HKCU) PSDrive.

```
s1 HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\Explorer\
```

2. Find the Key you're looking for:

```
Get-ChildItem -name A*
```

3. Find the Property (value) you want to change.

```
Get-ItemProperty .\Advanced\
```

4. Modify the item property

```
Set-ItemProperty -Path .\Advanced\ -PSProperty DontPrettyPath -  
value 0
```

Working with Registry (adding a key/value)

1. Change to the Registry (HKLM) PSDrive.

```
sl -path  
HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\OEMInformation
```

2. Create the Value:

```
New-ItemProperty -Path .\ -Name Logo -PropertyType String -value  
'C:\Media\me.bmp'
```

(creating a key would use `New-Item` instead of `new-itemproperty`)